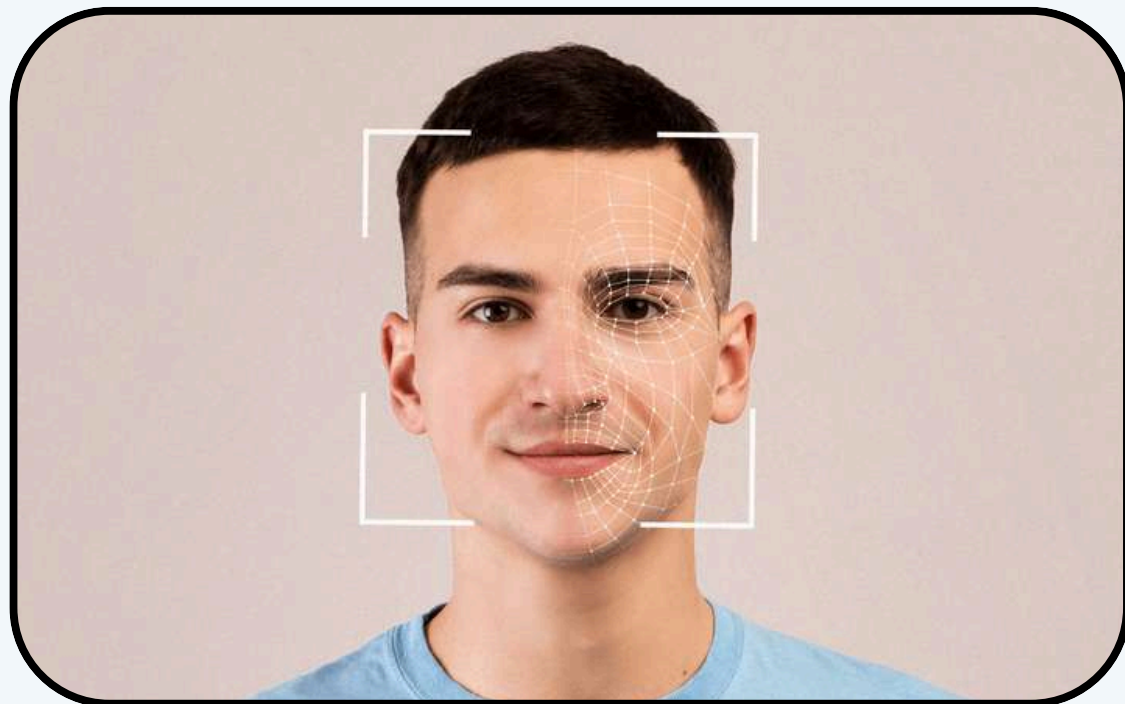


CORNER, LINE, DAN CIRCLE DETECTION

Menggunakan Citra Wajah (Face Detection)

Pengolahan Citra Medika (A)



**Jeremia Christ
Immanuel Manalu**

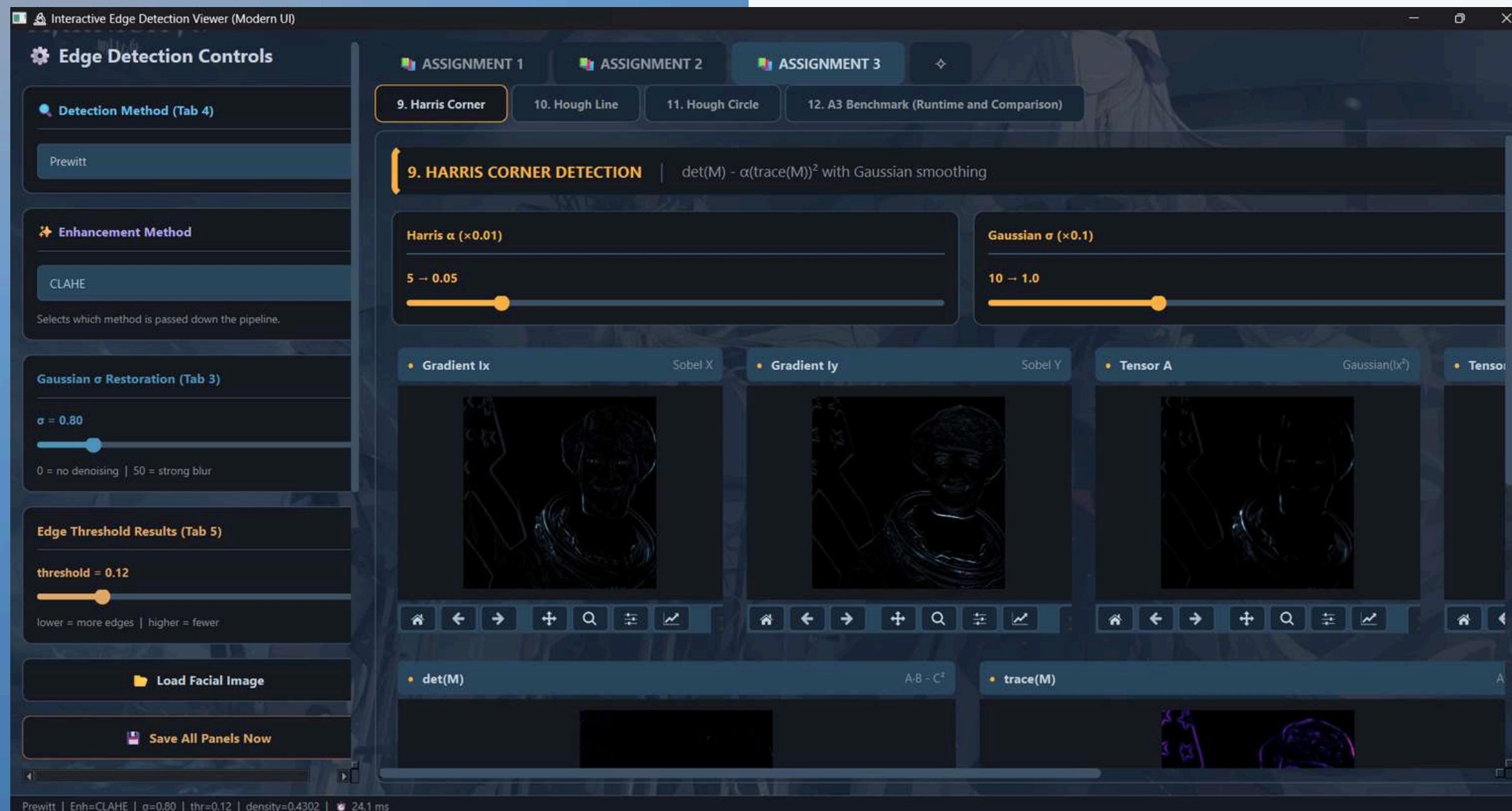


5023231017

01

PROGRAM

User Interface



```
root/
├── main.py
├── config.py
├── core/
│   ├── __init__.py
│   ├── math_ops.py
│   ├── enhancement.py
│   ├── edge_detection.py
│   ├── sharpening.py
│   ├── pipeline.py
│   └── feature_extractions.py
├── gui/
│   ├── __init__.py
│   ├── components.py
│   └── main_window.py
├── requirements.txt
└── README.md
```

Struktur Direktori

Hough Transform membutuhkan citra tepi biner. Program ini mengintegrasikan output Canny dari Assignment 2 secara dinamis sebagai input untuk Assignment 3.

Ketika slider Canny (σ , Low, High Threshold) di Tab A2 digeser, `_canny_debounce` akan langsung memicu ulang proses Hough Line dan Hough Circle di Tab A3.

Input yang masuk adalah matriks Hysteresis murni (from scratch), bukan library.

*Lanjutan dari Assignment 1, 2

HARRIS CORNER DETECTION

Mendeteksi area dengan variasi intensitas signifikan di berbagai arah (sudut) menggunakan turunan pertama (I_x, I_y).

01 Kalkulasi Gradien & Struktur Tensor (M)

Formula:

$$M = \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix} = \begin{pmatrix} A & C \\ C & B \end{pmatrix}$$

```
# 1. Gradients (Sobel)
Kx = KERNELS["Sobel"]["Gx"]
Ky = KERNELS["Sobel"]["Gy"]
Ix = manual_convolve2d(gray, Kx)
Iy = manual_convolve2d(gray, Ky)

# 2. Structure tensor elements
Ixx = Ix * Ix
Iyy = Iy * Iy
Ixy = Ix * Iy

# 3. Gaussian smoothing
A = manual_gaussian_filter(Ixx, sigma)
B = manual_gaussian_filter(Iyy, sigma)
C = manual_gaussian_filter(Ixy, sigma)
```

Gaussian Smoothing 02

Matriks A,B,C,A,B,C dihaluskan menggunakan `manual_gaussian_filter` untuk membuat detektor kebal terhadap noise (Isotropik).

HARRIS CORNER (RESPONSE DAN EVALUASI)

03 Kalkulasi Q-Value

Menghindari perhitungan eigenvalue yang mahal, Harris menggunakan penaksiran Determinan dan Trace.

$$Q = (A \cdot B - C^2) - \alpha(A + B)^2$$

```
# 4. det and trace
detM = A * B - C ** 2
trace = A + B

# 5. Harris response Q
Q = detM - alpha * (trace ** 2)
```

Visualisasi & Evaluasi 04

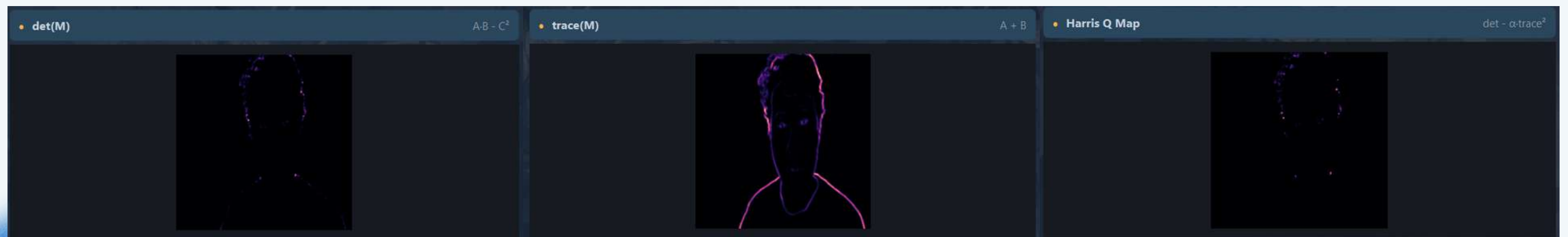
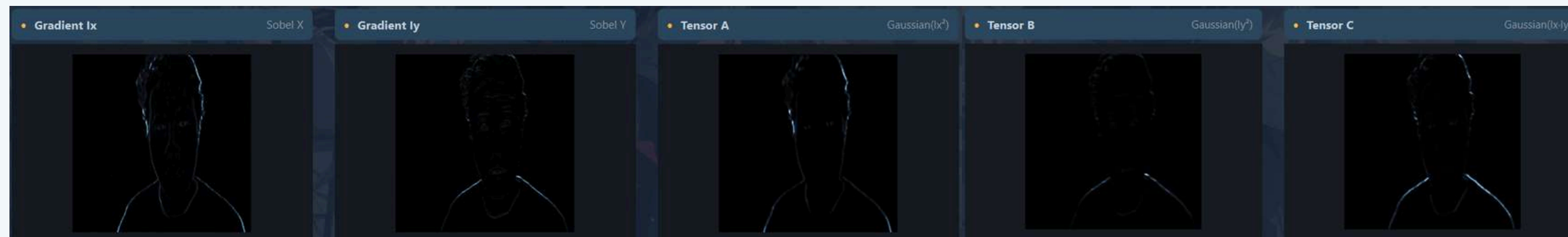
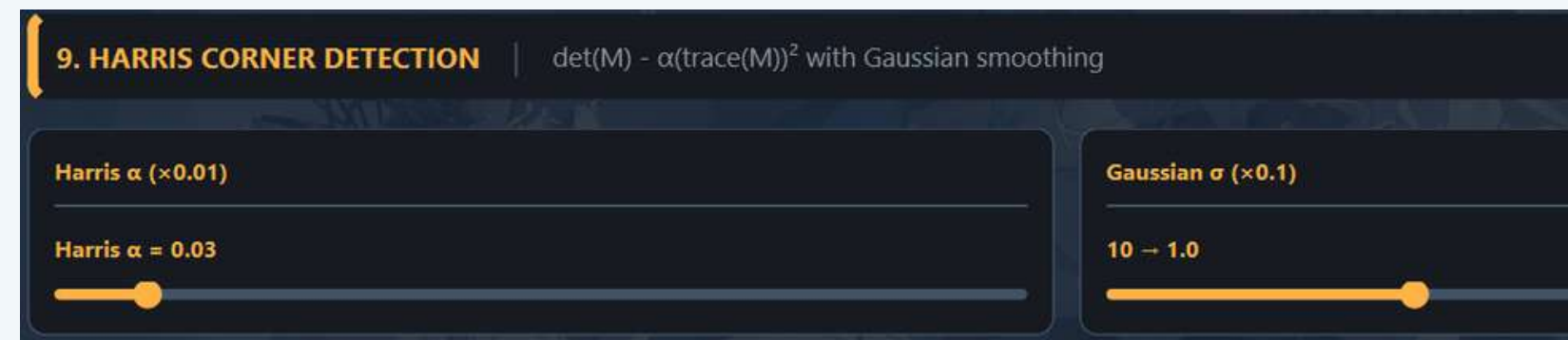
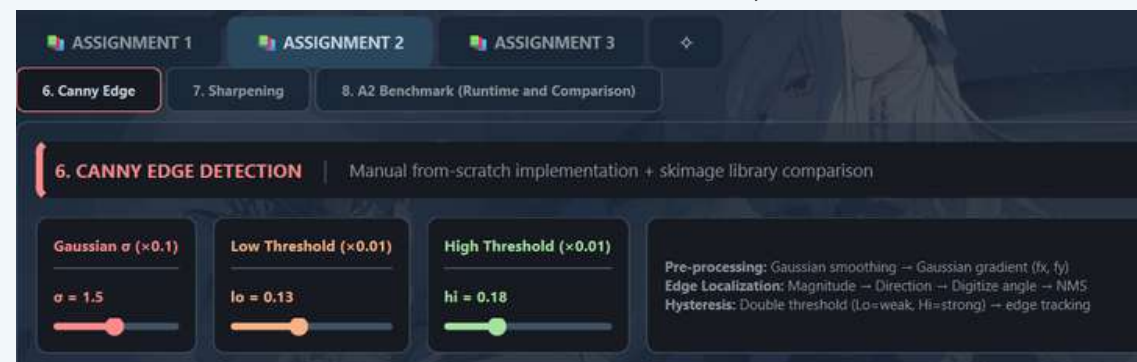
Jika Q positif dan melampaui threshold, area didefinisikan sebagai sudut (corner).

```
# 6. Threshold (relative to max positive Q)
Q_pos_max = Q[Q > 0].max() if (Q > 0).any() else 1.0
corners = Q > (threshold * Q_pos_max)

# Overlay: red corners on gray
overlay = _gray_to_rgb_uint8(gray)
_mark_corners_on_rgb(overlay, corners, color=(255, 50, 50), dot_r=2)
```

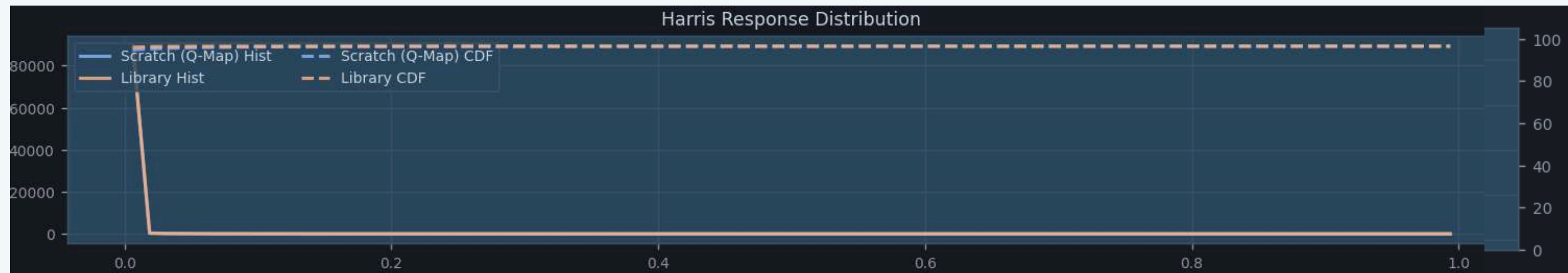
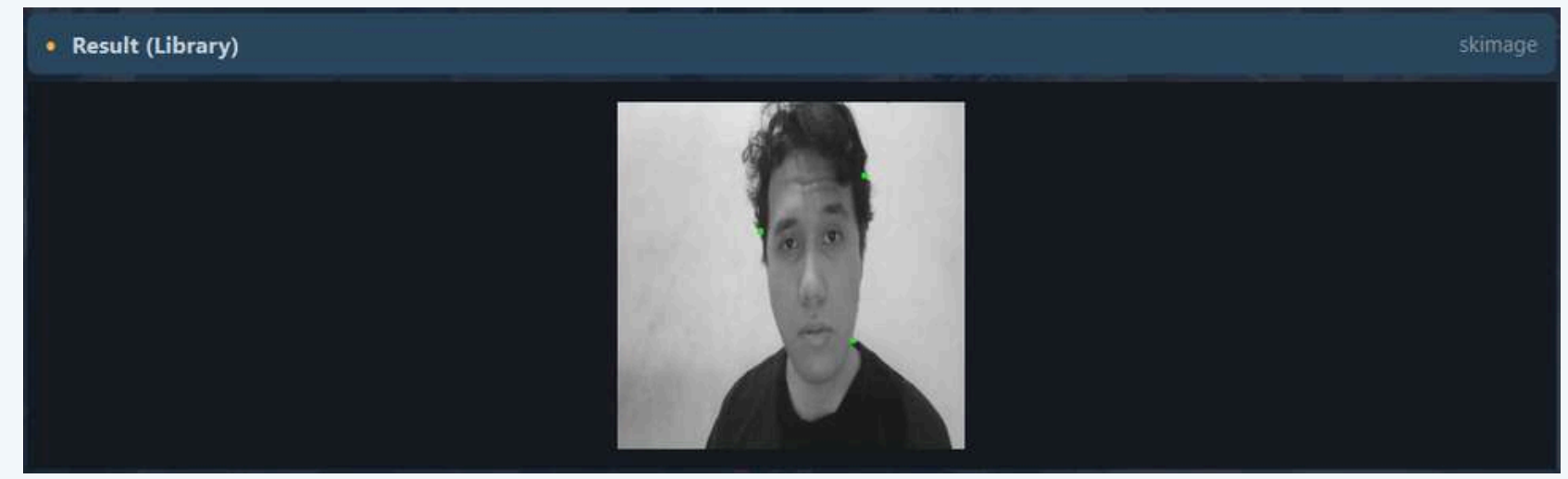

HARRIS CORNER (CONTOH DENGAN INPUT IMAGE SENDIRI)

04



HARRIS CORNER (CONTOH DENGAN INPUT IMAGE SENDIRI) CONT.

05



HOUGH LINE TRANSFORM

Mencari struktur garis lurus secara global (Gestalt theorem) dari edge map menggunakan Parameter Space.

01. Hessian Normal Form (HNF)

Menghindari nilai kemiringan tak terhingga ($k=\infty$) pada garis vertikal dengan menggunakan sistem koordinat polar (r, θ).

$$r = x \cdot \cos(\theta) + y \cdot \sin(\theta)$$

```
canny_out = canny_scratch(gray, sigma=c_sigma, t_lo=c_lo, t_hi=c_hi)
edges = (canny_out["hysteresis"] > 0.5) # Konversi ke binary boolean mask

H, W = edges.shape
x_r, y_r = W // 2, H // 2
theta_vals = np.linspace(0, np.pi, theta_steps, endpoint=False)
r_max = int(np.hypot(W, H)) + 1
accumulator = np.zeros((2 * r_max, theta_steps), dtype=np.int64)
ys, xs = np.nonzero(edges)

if len(xs) > 0:
    u = (xs - x_r)[:, np.newaxis].astype(np.float64)
    v = (ys - y_r)[:, np.newaxis].astype(np.float64)
    cos_t = np.cos(theta_vals)[np.newaxis, :]
    sin_t = np.sin(theta_vals)[np.newaxis, :]
    r_mat = u * cos_t + v * sin_t
    j_mat = np.round(r_mat).astype(int) + r_max
    valid = (j_mat >= 0) & (j_mat < 2 * r_max)
    k_mat = np.tile(np.arange(theta_steps), (len(xs), 1))
    np.add.at(accumulator, (j_mat[valid], k_mat[valid]), 1)
```

02. Accumulator Space (Voting)

Setiap piksel tepi (x, y) "memilih" kemungkinan garis di matriks akumulator. Titik persimpangan terbanyak (local maxima) diekstrak sebagai garis final.

HOUGH LINE TRANSFORM RESULT

Theta Steps

Theta Steps = 147

Threshold (Votes)

Threshold = 22

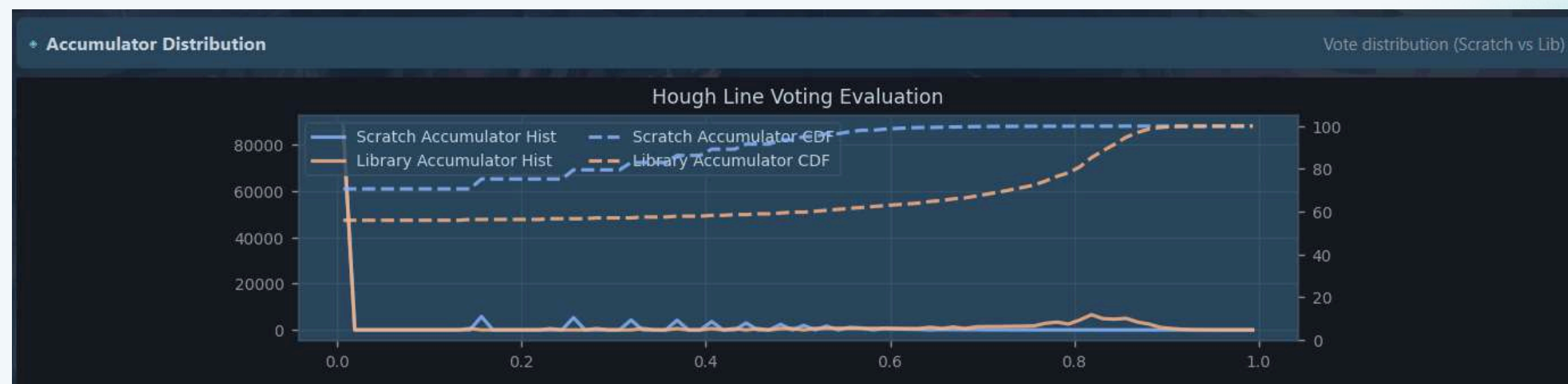
• Accumulator (Library) skimage.hough_line

• Detected Lines (Library) skimage.peaks

• Canny Edge Input for Hough

• Accumulator (Scratch) Log scale heatmap

• Detected Lines (Scratch) Manual drawing



HOUGH CIRCLE TRANSFORM

Modifikasi konsep Hough untuk mendeteksi objek berbentuk lingkaran.

01. Persamaan Lingkaran Parametrik

Mencari kandidat titik pusat (a,b) berdasarkan piksel tepi (x,y) dan radius yang ditentukan (r).

$$a = x - r \cdot \cos(\theta)$$

$$b = y - r \cdot \sin(\theta)$$

02. Eksekusi From Scratch

Program melakukan looping untuk sudut $\theta \in [0, 360)$ dengan loncatan 2 derajat guna mengisi Accumulator 2D untuk pencarian pusat lingkaran.

```
canny_out = canny_scratch(gray, sigma=c_sigma, t_lo=c_lo, t_hi=c_hi)
edges = (canny_out["hysteresis"] > 0.5) # Konversi ke binary boolean mask

H, W = edges.shape
accumulator = np.zeros((H, W), dtype=np.float64)
ys, xs = np.nonzero(edges)
```

```
if len(xs) > 0:
    t_deg = np.arange(0, 360, 2)
    cos_t, sin_t = np.cos(np.deg2rad(t_deg)), np.sin(np.deg2rad(t_deg))
    xs_col, ys_col = xs[:, np.newaxis].astype(np.float64), ys[:, np.newaxis].astype(np.float64)
    a_all = (xs_col - radius * cos_t[np.newaxis, :]).astype(int)
    b_all = (ys_col - radius * sin_t[np.newaxis, :]).astype(int)
    valid = (a_all >= 0) & (a_all < W) & (b_all >= 0) & (b_all < H)
    np.add.at(accumulator, (b_all[valid], a_all[valid]), 1)

acc_max = accumulator.max()
centers = np.argwhere(accumulator > (threshold_frac * acc_max if acc_max > 0 else 1.0))
result_rgb = _gray_to_rgb_uint8(gray)
for cy, cx in centers[:40]:
    _draw_circle_on_rgb(result_rgb, cx, cy, radius, H, W, color=(0, 220, 0))

acc_disp = accumulator / acc_max if acc_max > 0 else accumulator
```


HOUGH CIRCLE TRANSFORM RESULT

Radius (px) [Scratch]

Radius = 24

Threshold Fraction ($\times 0.01$)

Thr Frac = 0.98

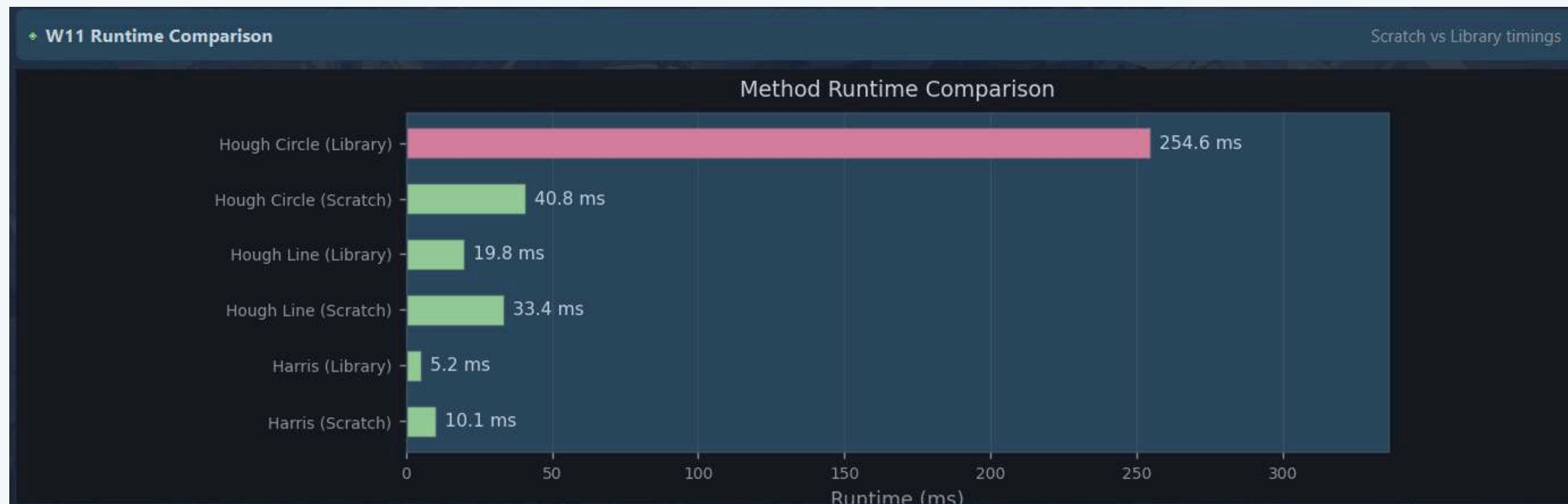


EVALUASI AKUMULATOR (HOUGH ANALYSIS)

- **Log-scale Visualization:** Nilai akumulator direntangkan menggunakan `np.log1p()` agar persimpangan garis (votes tertinggi) bersinar layaknya heatmap.
- **Histogram Comparison:** Plot distribusi membuktikan bahwa akumulasi voting dari algoritma scratch memiliki kecenderungan puncak (peak) yang sepadan dengan versi skimage (Library).

RUN-TIME ANALYSIS DAN BENCHMARK

- **Harris Corner:** Cenderung cepat karena berbasis perkalian matriks linear dan Gaussian.
- **Hough Transform (Scratch vs Library):** Algoritma from scratch yang menggunakan NumPy array (vectorized) cukup responsif, namun versi Library (skimage) jauh lebih cepat karena berjalan di atas optimalisasi lapisan Cython/C++ tingkat rendah.



DAN CONTOH-CONTOH LAIN

<https://its.id/m/Tugas3CornerLineCircleDetectionJemi>

Catatan:

- Pada setiap sub-tab bisa diubah parameter-parameternya sendiri.

Kesimpulan

Ekstraksi fitur spasial (Sudut, Garis, Lingkaran) berhasil dibangun sepenuhnya menggunakan operasi matriks mandiri (from scratch).

Kualitas Hough Transform sangat bergantung pada kualitas ekstraksi Canny Edge. Konektivitas parameter Canny (Tab 2) ke input Hough (Tab 3) terbukti mempermudah tuning (parameter sweeping).

Evaluasi ruang akumulator membuktikan bahwa deteksi shapes dapat direpresentasikan secara matematis sebagai masalah pencarian nilai local maxima di dalam matriks parameter polar.



THANK YOU

